



Eco PR 354

Security Review

Cantina Managed review by:

Rikard Hjort, Lead Security Researcher

0xHuy0512, Security Researcher

February 5, 2026

Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
2.1	Scope	3
3	Findings	4
3.1	Medium Risk	4
3.1.1	Solver can grief refunds by setting a bad claimant	4
3.2	Low Risk	4
3.2.1	Duplicate tokens in route causes <code>flashFulfill()</code> to fail due to overwritten approvals	4
3.2.2	<code>reward.prover</code> can be different from the currently executing <code>LocalProver</code>	5
3.3	Gas Optimization	5
3.3.1	<code>_flashFulfillInProgress</code> can be made transient	5
3.4	Informational	6
3.4.1	Incorrect comment	6

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Eco enables apps to unlock stablecoin liquidity from any connected chain and give users the simplest onchain experience.

From Jan 17th to Jan 19th the Cantina team conducted a review of [eco-routes](#) on commit hash [bc1097b3](#). The team identified a total of **5** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	1	1	0
Low Risk	2	2	0
Gas Optimizations	1	1	0
Informational	1	1	0
Total	5	5	0

2.1 Scope

The security review had the following components in scope for [eco-routes](#) on commit hash [bc1097b3](#):

```
contracts
├── interfaces
│   └── ILocalProver.sol
├── prover
│   └── LocalProver.sol
```

3 Findings

3.1 Medium Risk

3.1.1 Solver can grief refunds by setting a bad claimant

Severity: Medium Risk

Context: [IntentSource.sol#L798-L813](#), [Vault.sol#L102](#)

Summary: The solver can opt to forego rewards in exchange for also blocking refunds, keeping funds stuck in a Vault indefinitely.

Finding Description: In `Vault.sol`, part of the withdrawal process is sending native tokens to the claimant via `call`. This can fail if the solver has chosen a contract as claimant (or delegated the receiving address to a contract via a EIP-7702 delegation) and that contract reverts. The solver could do so maliciously. The result is that withdrawals will always fail, and thus, so will refunds, because refunds depends on the intent either being unfulfilled or withdrawn.

There seems to have been an assumption that solvers cannot block withdrawals because `withdraw()` is callable by anyone. But this assumption fails to take grieving into account.

Impact Explanation: The creator can not receive any refunds on an intent. They will, however, have the intent fulfilled. Thus the impact is limited to any additional funds that have been submitted to the Vault.

Likelihood Explanation: This attack works on any intent which specifies a non-zero native reward amount. It is profitable or meaningful only for an Vault with an oversupply of rewards. The solver does not benefit from the attack, they only lose by having to perform the complete fulfillment of the intent and foregoing the reward. However, the attack is technically easy to perform and can be triggered even after fulfillment, for any number of Vaults simultaneously, by choosing a claimant address which they have a private key for and EIP7702-delegating, or by having it be a contract address which is already deployed or which they deploy at a later time, which reverts for any reason.

Recommendation: Use Solady `forceTransferETH()` or similar to prevent the distribution of ETH to fail.

If the chain does not support `selfdestruct` as needed by `forceSafeTransferETH()`, simply remove the reverting condition. If the call fails, leave the native tokens in the contract.

Eco Foundation: Fixed in [PR 360](#). For the chains that don't support `selfdestruct`, `try/catch` gets around it by just returning ETH to the creator if the claimant can't take it.

Cantina Managed: Fix verified.

3.2 Low Risk

3.2.1 Duplicate tokens in route causes `flashFulfill()` to fail due to overwritten approvals

Severity: Low Risk

Context: [LocalProver.sol#L196-L201](#)

Description: In `LocalProver.flashFulfill()`, the contract approves the portal (`_PORTAL`) to spend tokens before calling `_PORTAL.fulfill()`. The approval is done in a loop iterating over `route.tokens`:

```
for (uint256 i = 0; i < tokensLength; ++i) {
    IERC20(route.tokens[i].token).approve(
        address(_PORTAL),
        route.tokens[i].amount
    );
}
```

The root cause is that `approve()` sets an absolute allowance rather than incrementing it. If the same token address appears multiple times in `route.tokens[]`, each subsequent `approve()` call overwrites the previous approval amount instead of accumulating them.

When `_PORTAL.fulfill()` is subsequently called, the `Inbox._fulfill()` function executes `safeTransferFrom()` for each token entry. Since the approval only reflects the last occurrence's amount

rather than the cumulative total, the transfer will revert with insufficient allowance when attempting to transfer duplicate token entries.

Note that this issue only affects `flashFulfill()`. In the normal `fulfill()` case where the solver directly calls the portal, tokens are transferred directly from `msg.sender` without intermediate approvals, so duplicate tokens are handled correctly.

Recommendation: Replace `approve()` with `safeIncreaseAllowance()` from OpenZeppelin's `SafeERC20` library to accumulate allowances for duplicate token addresses.

Eco Foundation: Fixed in PR 358.

Cantina Managed: Fix verified.

3.2.2 `reward.prover` can be different from the currently executing `LocalProver`

Severity: Low Risk

Context: `LocalProver.sol`#L172

Description: It is possible to call `flashFulfill()` with an intent specifying a different prover than the `LocalProver`. In that case, `provenIntents()` will be called on some other contract, creating a different flow. The user can set this to be any contract of their choosing, including a malicious one which can return anything on a `provenIntents()` call. This creates a hard-to-reason-about scenario and a potential footgun. However, there are no known vulnerabilities caused by this.

Recommendation: Add a check to ensure the only flow in `flashFulfill()` is the expected one, with a callback to the prover itself:

```
require(reward.prover == address(this));
```

Eco Foundation: Fixed in PR 359.

Cantina Managed: Fix verified.

3.3 Gas Optimization

3.3.1 `_flashFulfillInProgress` can be made transient

Severity: Gas Optimization

Context: `LocalProver.sol`#L189

Description: `_flashFulfillInProgress` is set to the currently flash-fulfilling intent hash in order to create special behavior in `provenIntents()`. The same intent can not be fulfilled twice, and `flashFulfill()` cannot be reentered, so the same intent will not have `provenIntents()` called on it twice by the actual portal. Therefore the storage variable does not strictly need to be cleared after each transaction.

However, leaving it set to a valid intent hash slightly complicates reasoning about attack patterns, and it would be recommended to reset it to some safe default value after each transaction, e.g. `0` or `1`, which will not be valid intent hashes.

This is an excellent use case for transient storage, which is cheaper to write to than regular storage, and is reset after each transaction.

Recommendation: Verify that all chains you are using support transient storage. If they do not, clear `_flashFulfillInProgress` at the end of each transaction, for example to a simply chosen but non-zero value like `bytes32(1)`, to avoid code smell. Then:

In `LocalProver.sol`:

```
// SPDX-License-Identifier: MIT
- pragma solidity ^0.8.13;
+ pragma solidity ^0.8.28;

import {SafeCast} from "@openzeppelin/contracts/utils/math/SafeCast.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
// ...
```

```

    * @dev Used to enable withdrawal during flashFulfill execution (before
    ↳ Portal.claimants is set)
    * Only one flashFulfill can execute at a time due to nonReentrant modifier
    */
- bytes32 private _flashFulfillInProgress;
+ bytes32 transient private _flashFulfillInProgress;

constructor(address portal) {
    _PORTAL = IPortal(portal);
}

```

In foundry.toml:

```

[profile.default]
- solc_version="0.8.27"
+ solc_version="0.8.28"
src = "contracts"
out = "out"
fs_permissions = [{ access = "read-write", path = "./out"}]
# ...
optimizer_runs = 1000000
via_ir = true
retries = 2
- evm_version = "paris"
+ evm_version = "cancun"
cbor_metadata = false
bytecode_hash = "none"

```

Eco Foundation: Fixed in [PR 356](#).

Cantina Managed: Fix verified.

3.4 Informational

3.4.1 Incorrect comment

Severity: Informational

Context: [LocalProver.sol#L213](#)

Description: The linked comment marks tokens transfers as "effects" in the "Checks, Effects, Interactions" pattern. They are interactions, not effects. However, they are still in the correct location in the code -- after all the effects.

Recommendation:

```

- // EFFECTS - Transfer remaining funds to claimant
+ // INTERACTIONS - Transfer remaining funds to claimant

```

Eco Foundation: Fixed in [PR 357](#).

Cantina Managed: Fix verified.